

Analisis de seguridad

Proyecto: Total Secure App

Version: 1.0

Fecha: 15/09/2026

Elaborado por: Auditoria tecnica

Revision del **2026-09-15**. Ningun cambio aplicado.

⚠ **Contexto:** es el **unico sistema del servidor expuesto a internet** y el que mas datos personales sensibles almacena: **12.664 registros biometricos** de 880 usuarios.

⚠ Revision ampliada del 2026-09-15 (segunda pasada)

Al documentar la API ruta por ruta aparecio **una toma de cuentas**: un endpoint publico que cambia la contrasena de cualquier usuario sin validar ningun token. No se habia detectado en la primera pasada, que reviso configuracion y exposicion pero no el codigo de cada controlador.

Resumen

Severidad	Cantidad
● CRITICO	2
● ALTO	2
● MEDIO	4
● BAJO	1
○ INFORMATIVO	3

● CRITICO

SEC-00 — Toma de cuentas: cambio de contrasena sin autenticacion ni token

Evidencia. `POST /api/procesar_paswchg` tiene middleware `['api']`, sin autenticacion (confirmado en `php artisan route:list --json`). Su controlador, `Modules/MobileApp/Http/Controllers/LoginController.php`:

```
$user_id    = $request->user_id;
$password   = $request->password;
$rsUsuario = users::find($user_id);
// ... valida solo el FORMATO de la contrasena ...
$rsUsuario->usu_password   = $password;
$rsUsuario->remember_token = "";
$rsUsuario->save();
```

El `remember_token` que emite `solicitud_cambiopass` nunca se comprueba. Basta `user_id` y la contrasena nueva.

Impacto. Cualquiera que alcance el endpoint —y el sistema responde desde internet— toma control de **cualquiera de las 880 cuentas**, incluidas las

administrativas, con una sola petición. Los `user_id` son enteros secuenciales.

Desde una cuenta administrativa se alcanzan **12.664 registros biometricos**, 9.769 accesos y 8.667 rondas: datos personales sensibles que no se pueden cambiar si se filtran.

Agravante. `POST /api/solicitud_paswchg`, también público, **devuelve el token y el `user_id` en la respuesta**, y lo genera con `rand(1000, 10000000)` — no criptográfico. Aunque se corrigiera la validación, esto por sí solo permitiría el ataque.

Atenuante. El modelo `users` cifra la contraseña al guardar (evento `saving` → `Hash::make`), así que **no se guardan en claro**. Limita el daño posterior, no la toma de la cuenta.

No se ejecutó la prueba: habría cambiado la contraseña de un usuario real en producción. La evidencia de código y el middleware vacío son concluyentes.

Recomendación — es lo primero de todo el proyecto, por delante de HTTPS:

1. Validar el `remember_token` contra el usuario y su vigencia.
2. Generarlo con `random_bytes`, no con `rand()`.
3. **No devolverlo** en la respuesta de `solicitud_paswchg`.
4. Invalidarlo tras el primer uso.

Mientras no se corrija, **cerrar el acceso público a esas dos rutas** es una mitigación inmediata y válida.

SEC-01 — Biometría y credenciales viajando por internet en HTTP plano

Evidencia, en tres piezas que se refuerzan:

1. **El sistema responde desde internet.** Comprobado:
`curl http://181.198.245.50:3031/` → **HTTP 302**. No es exposición de red local: es la internet pública.
2. **El backend se configura con esa IP pública sobre HTTP:**
`APP_URL: http://181.198.245.50:3031` en `docker-compose.yml`. Sin TLS.
3. **La aplicación móvil habilita tráfico sin cifrar de forma explícita:**
`app.json` declara `"usesCleartextTraffic": true`. Android bloquea HTTP por defecto desde la versión 9; aquí se desactiva esa protección a propósito.

Impacto: todo lo que intercambian las tablets de los guardias y el servidor viaja **legible para cualquiera en el camino**: contraseñas, tokens de sesión, registros de acceso, ubicaciones de ronda y **datos biometricos**. Quien comparta la red de un guardia —el wifi de un edificio vigilado, por ejemplo— puede leerlo y modificarlo.

La biometria agrava el problema: una contraseña filtrada se cambia; **una huella o un rostro, no.**

Recomendacion: TLS. Existe `docker-compose.prod.yml` con nginx 1.27 y **certbot** ya previstos (puertos 80 y 443), y `DESPLIEGUE-DOMINIO.md` documenta el despliegue con dominio. **La solucion ya esta preparada en el proyecto y no se activo.** Una vez con HTTPS, quitar `usesCleartextTraffic` y recompilar el APK.



SEC-02 — El codigo fuente se monta dentro de los contenedores

Evidencia: `docker-compose.yml` monta `./:/var/www` en **backend y nginx**.

Impacto: los contenedores ejecutan lo que haya en el disco del host, no una imagen inmutable. Un cambio en el directorio altera produccion sin reconstruir ni dejar registro. Y el contenedor puede escribir sobre el codigo: ante una vulnerabilidad de ejecucion remota, el atacante modifica el fuente directamente.

En un sistema expuesto a internet, pesa mas que en uno interno.

Recomendacion: imagen con el codigo copiado (`COPY`) para produccion.

SEC-03 — Keystore de firma dentro del proyecto

Evidencia: `apk/totalsecureapp-release.jks` , permisos `600` .

SECRET DETECTADO — NO DOCUMENTAR VALOR.

Impacto: ese archivo **es** la identidad de la aplicacion. Quien lo obtenga puede firmar un APK que las tablets aceptaran como actualizacion legitima, y no hay forma de revocarlo: habria que publicar la app con otra identidad y reinstalarla en todos los dispositivos.

Atenuantes, verificados: permisos correctos (`600`);

`LEEME-keystore.txt` documenta su manejo; y **el** `.gitignore` **SI lo excluye** (`/apk/` y `*.jks`). El propio `.gitignore` registra que fue una correccion consciente:

".apk cubria el APK pero NO el keystore (...) git add -A publicaba la clave de firma en GitHub. Un keystore filtrado..."*

O sea que el riesgo de publicarlo por git **ya esta cerrado**. Lo que queda es el riesgo de perderlo.

Recomendacion: sacarlo del directorio del proyecto a un almacen de secretos, y conservar copia fuera del servidor. Sin el keystore **no se pueden publicar actualizaciones**, asi que perderlo es tan grave como filtrarlo.

● MEDIO

SEC-04 — Sin respaldo automatico verificado

No se encontro cron ni timer para este proyecto. Con 57 tablas, 880 usuarios y 12.664 biometrias, es el sistema con mas que perder. Ver `12_CONTINUIDAD`.

SEC-05 — Aislamiento entre instituciones sin verificar

Con ~38.247 vinculos usuario-institucion, **que un usuario no vea datos de una institucion que no le corresponde es una regla critica**. Usa `spatie/laravel-permission`, que es la libreria correcta, pero **la verificacion efectiva no se ejecuto**: `NO DETERMINADO`. Es la prueba pendiente mas importante.

SEC-06 — Contenedor `v1_analisis` suelto y sin control

MariaDB 10.4 (**version antigua**), creado el 2026-09-07, **sin proyecto compose**, con la base V1 heredada. No aparece en ningun `docker-compose.yml`, asi que **no se recrea ni se respalda con el resto**.

Atenuante: no publica puertos; solo es accesible desde la red Docker.

Recomendacion: definir si la migracion concluyo. Si si, respaldar su contenido y apagarlo.

● BAJO

SEC-07 — MariaDB 10.4 sin soporte

`v1_analisis` corre MariaDB 10.4, una version fuera de soporte. Al no estar expuesta, el riesgo es acotado, pero es una razon mas para apagarla.

○ INFORMATIVO

SEC-08 — PostgreSQL correctamente limitado

`ports: "127.0.0.1:5434:5432"` — **solo local**, a diferencia de otros proyectos del servidor que publican su base a toda interfaz. Correcto.

SEC-09 — DB_PASSWORD con fail-fast

`${DB_PASSWORD:?Definir DB_PASSWORD en .env}` en los dos servicios.

`SECRET DETECTADO – NO DOCUMENTAR VALOR.`

✓ Hallazgo previo YA CORREGIDO — vale la pena registrarlo

El `docker-compose.yml` documenta en un comentario extenso un incidente real:

"Estaba en local con APP_DEBUG: 'true', y esto es produccion expuesta en dos IP publicas. Con el modo depuracion encendido, cualquier error 500 devuelve la traza completa (...) incluida la contraseña de la base."

Verificado hoy: el contenedor corre con `APP_ENV=production` y `APP_DEBUG=false`. **La fuga esta cerrada.**

El comentario ademas advierte algo que conviene no olvidar: "estos valores pisan al .env, asi que cambiar solo el archivo no alcanza". Es exacto, y es la clase de trampa que hace perder horas.

✓ El webhook de WhatsApp, bien resuelto

Contrasta con el resto y merece registrarse.

`POST /api/whatsapp/webhook/{token}` compara el token con `hash_equals`

(resistente a ataques de tiempo) y responde **404** si no hay token configurado.

El comentario del codigo explica la decision: "Sin token configurado el webhook no existe: es una puerta abierta a que cualquiera simule respuestas de guardias."

Es la mejor implementacion de webhook del servidor — comparar con la de `lavado_de_carros`, que no valida firma.

Lo que se reviso y esta BIEN

- **PostgreSQL solo en** `127.0.0.1`.
- **DB_PASSWORD obligatoria con fail-fast** en ambos servicios.
- `APP_DEBUG=false` y `APP_ENV=production` verificados en ejecucion.
- **Healthcheck en la base** con `pg_isready`, y el backend esperando con `condition: service_healthy`.
- `spatie/laravel-permission` para roles, y `laravel/sanctum` para tokens de API: las librerias correctas, no implementaciones propias.
- **El keystore tiene permisos 600** y su manejo esta documentado.
- **La huella SHA-256 del certificado esta publicada** en `LEEME-apk.txt`, lo que permite verificar que un APK salio de este servidor. Es una practica que no se ve en ningun otro proyecto auditado.

- **El login es solido:** `Hash::check` , estado de cuenta, gestion asignada y rol activo, antes de emitir un token de Sanctum con expiracion y un refresh token de `random_bytes(32)` .
- **Solo 4 rutas publicas de 55**, y dos de ellas lo son legitimamente.
- `CheckPermission` **con permisos granulares** en biometria y en el portal.